
Algorithm 12 FMSCONTROLBITS(T)

Input: Secret routing-tag array T of public length $n = 2^h$
Output: Control-word array C containing one two-bit word for every OFork gate

- 1: $n \leftarrow |T|$
- 2: $\ell_C \leftarrow \text{FMSCONTROLNUM}(n)$
- 3: Create array C of length ℓ_C
- 4: $\hat{T} \leftarrow \text{TAGNORMALIZE}(T)$
- 5: $p \leftarrow \text{FMSCONTROLWRITE}(\hat{T}, C, 0)$
- 6: **assert** $p = \ell_C$
- 7: **return** C

Algorithm 13 FMSCONTROLNUM(n)

Input: Public node size $n = 2^h$, where $n \geq 2$
Output: Number ℓ of two-bit OFork control words

- 1: **if** $n = 2$ **then**
- 2: **return** 1
- 3: **end if**
- 4: $\ell_g \leftarrow n/2$
- 5: $\ell_{\text{sub}} \leftarrow \text{FMSCONTROLNUM}(\ell_g)$
- 6: **return** $\ell_g + 2\ell_{\text{sub}}$

Algorithm 14 FMSCONTROLWRITE(T, C, p)

Input: Normalized secret routing-tag array T of public length $n = 2^h$, where $\sum_i T[i][0] = \sum_i T[i][1] = n/2$, global control array C , and public writing position p
Output: Next free position p in C

- 1: $n \leftarrow |T|$
- 2: **if** $n = 2$ **then**
- 3: $C[p] \leftarrow \text{OFORKCONTROL}(T[0], T[1], 10, 01)$
- 4: **return** $p + 1$
- 5: **end if**
- 6: $\ell_g \leftarrow n/2$
- 7: $(C_{\text{cur}}, T_{\text{top}}, T_{\text{bot}}) \leftarrow \text{FMSBALANCE}(T)$
- 8: $C[p : p + \ell_g] \leftarrow C_{\text{cur}}$
- 9: $p \leftarrow p + \ell_g$
- 10: $p \leftarrow \text{FMSCONTROLWRITE}(T_{\text{top}}, C, p)$
- 11: $p \leftarrow \text{FMSCONTROLWRITE}(T_{\text{bot}}, C, p)$
- 12: **return** p

Algorithm 15 TAGPAIRMASK(x, y)

Input: Two secret routing tags x and y
Output: A one-hot array $M \in \{0, 1\}^{10}$ for the unordered tag-pair types $\{00, 00\}, \{00, 10\}, \{00, 01\}, \{00, 11\}, \{10, 10\}, \{10, 01\}, \{10, 11\}, \{01, 01\}, \{01, 11\}$, and $\{11, 11\}$, in this order

- 1: $M[0] \leftarrow \llbracket x = 00 \rrbracket \wedge \llbracket y = 00 \rrbracket$
- 2: $M[1] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 10 \rrbracket) \vee (\llbracket x = 10 \rrbracket \wedge \llbracket y = 00 \rrbracket)$
- 3: $M[2] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 01 \rrbracket) \vee (\llbracket x = 01 \rrbracket \wedge \llbracket y = 00 \rrbracket)$
- 4: $M[3] \leftarrow (\llbracket x = 00 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 00 \rrbracket)$
- 5: $M[4] \leftarrow \llbracket x = 10 \rrbracket \wedge \llbracket y = 10 \rrbracket$
- 6: $M[5] \leftarrow (\llbracket x = 10 \rrbracket \wedge \llbracket y = 01 \rrbracket) \vee (\llbracket x = 01 \rrbracket \wedge \llbracket y = 10 \rrbracket)$
- 7: $M[6] \leftarrow (\llbracket x = 10 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 10 \rrbracket)$
- 8: $M[7] \leftarrow \llbracket x = 01 \rrbracket \wedge \llbracket y = 01 \rrbracket$
- 9: $M[8] \leftarrow (\llbracket x = 01 \rrbracket \wedge \llbracket y = 11 \rrbracket) \vee (\llbracket x = 11 \rrbracket \wedge \llbracket y = 01 \rrbracket)$
- 10: $M[9] \leftarrow \llbracket x = 11 \rrbracket \wedge \llbracket y = 11 \rrbracket$
- 11: **return** M

Algorithm 16 TAGNORMALIZE(T)

Input: Secret routing-tag array T of public length $n = 2^h$, where $\sum_{i=0}^{n-1} T[i][0] \leq n/2$ and $\sum_{i=0}^{n-1} T[i][1] \leq n/2$
Output: A normalized routing-tag array $\hat{T}$ satisfying $\sum_{i=0}^{n-1} \hat{T}[i][0] = \sum_{i=0}^{n-1} \hat{T}[i][1] = n/2$ and every nonzero input tag remains unchanged

- 1: $n \leftarrow |T|$
- 2: $w_L \leftarrow 0, w_R \leftarrow 0$ $\triangleright$ Current left and right responsibilities
- 3: **for** $i \leftarrow 0 \dots n - 1$ **do**
- 4: $w_L \leftarrow w_L + T[i][0]$
- 5: $w_R \leftarrow w_R + T[i][1]$
- 6: **end for**
- 7: $a_L \leftarrow n/2 - w_L$ $\triangleright$ Number of 00 tags assigned to the left
- 8: $a_R \leftarrow n/2 - w_R$ $\triangleright$ Number of 00 tags assigned to the right
- 9: Create an array $\hat{T}$ of length n
- 10: $r_0 \leftarrow 0$ $\triangleright$ Number of preceding 00 tags
- 11: **for** $i \leftarrow 0 \dots n - 1$ **do**
- 12: $z \leftarrow \llbracket T[i] = 00 \rrbracket$
- 13: $u_L \leftarrow z \wedge \llbracket r_0 < a_L \rrbracket$
- 14: $u_R \leftarrow z \wedge \llbracket a_L \leq r_0 \rrbracket \wedge \llbracket r_0 < a_L + a_R \rrbracket$
- 15: $\hat{T}[i] \leftarrow T[i]$
- 16: $\hat{T}[i] \leftarrow \text{OSELECT}(\hat{T}[i], 10, u_L)$
- 17: $\hat{T}[i] \leftarrow \text{OSELECT}(\hat{T}[i], 01, u_R)$
- 18: $r_0 \leftarrow r_0 + z$
- 19: **end for**
- 20: **return** $\hat{T}$

Algorithm 17 OFORKCONTROL($x, y, t_{\text{top}}, t_{\text{bot}}$)

Input: Input tags x, y and desired secret output tags $t_{\text{top}}, t_{\text{bot}}$

Output: A two-bit OFORK control word c

```
1:  $m_{\text{straight}} \leftarrow \llbracket t_{\text{top}} = x \rrbracket \wedge \llbracket t_{\text{bot}} = y \rrbracket$ 
2:  $m_{\text{cross}} \leftarrow \llbracket t_{\text{top}} = y \rrbracket \wedge \llbracket t_{\text{bot}} = x \rrbracket$ 
3:  $f_x \leftarrow \llbracket x = 11 \rrbracket \wedge \llbracket y = 00 \rrbracket$ 
4:  $f_y \leftarrow \llbracket x = 00 \rrbracket \wedge \llbracket y = 11 \rrbracket$ 
5:  $f \leftarrow f_x \vee f_y$ 
6:  $m_{\text{LR}} \leftarrow \llbracket t_{\text{top}} = 10 \rrbracket \wedge \llbracket t_{\text{bot}} = 01 \rrbracket$ 
7:  $m_{\text{RL}} \leftarrow \llbracket t_{\text{top}} = 01 \rrbracket \wedge \llbracket t_{\text{bot}} = 10 \rrbracket$ 
8:  $m_{\text{fork}} \leftarrow f \wedge (m_{\text{LR}} \vee m_{\text{RL}})$ 
9:  $c_{\text{fork}} \leftarrow (f_y, f_x) \quad \triangleright (0,0) \text{ copies } x; (1,1) \text{ copies } y$ 
10:  $u_{\text{cross}} \leftarrow m_{\text{cross}} \wedge \neg m_{\text{straight}}$ 
11:  $u_{\text{fork}} \leftarrow m_{\text{fork}} \wedge \neg m_{\text{straight}} \wedge \neg m_{\text{cross}}$ 
12:  $c \leftarrow (0, 1)$ 
13:  $c \leftarrow \text{OSELECT}(c, (1, 0), u_{\text{cross}})$ 
14:  $c \leftarrow \text{OSELECT}(c, c_{\text{fork}}, u_{\text{fork}})$ 
15: return  $c$ 
```

Algorithm 18 FMSAPPLY(D, C, p)

Input: Current data array D of public length $n = 2^h$, global control-word array C , and public reading position p

Output: Left and right output arrays D_L, D_R of length $n/2$, and next unread position p in C

```
1:  $n \leftarrow |D|$ 
2: if  $n = 2$  then
3:    $(d_L, d_R) \leftarrow \text{OFORK}(D[0], D[1], C[p])$ 
4:   return  $([d_L], [d_R], p + 1)$ 
5: end if
6:  $\ell_g \leftarrow n/2$ 
7: Create length- $\ell_g$  arrays  $D_{\text{top}}$  and  $D_{\text{bot}}$ 
8: for  $i \leftarrow 0 \dots \ell_g - 1$  do
9:    $(D_{\text{top}}[i], D_{\text{bot}}[i]) \leftarrow \text{OFORK}(D[2i], D[2i + 1], C[p + i])$ 
10: end for
11:  $p \leftarrow p + \ell_g$ 
12:  $(A_L, A_R, p) \leftarrow \text{FMSAPPLY}(D_{\text{top}}, C, p)$ 
13:  $(B_L, B_R, p) \leftarrow \text{FMSAPPLY}(D_{\text{bot}}, C, p)$ 
14:  $D_L \leftarrow A_L \parallel B_L$ 
15:  $D_R \leftarrow A_R \parallel B_R$ 
16: return  $(D_L, D_R, p)$ 
```

Algorithm 19 FMSBALANCE(T)

Input: Secret routing-tag array T of public length $n \geq 4$, where $\sum_i T[i][0] = \sum_i T[i][1] = n/2$

Output: Current-layer control array C_{cur} and balanced child-tag arrays T_{top} and T_{bot}

```
1:  $n \leftarrow |T|$ 
2:  $\ell_g \leftarrow n/2$ 
3: Create array  $H$  of length 10, initialized to 0  $\triangleright$  Tag-pair type counts
4: for  $i \leftarrow 0 \dots \ell_g - 1$  do
5:    $x \leftarrow T[2i], y \leftarrow T[2i + 1]$ 
6:    $M \leftarrow \text{TAGPAIRMASK}(x, y)$ 
7:   for  $j \leftarrow 0 \dots 9$  do
8:      $H[j] \leftarrow H[j] + M[j]$ 
9:   end for
10: end for
11:  $d \leftarrow H[5] \quad \triangleright$  Number of  $\{10, 01\}$  pairs
12:  $v_L \leftarrow H[1] + H[3] + H[8]$ 
13:  $v_R \leftarrow H[2] + H[3] + H[6]$ 
14:  $q_D \leftarrow \lfloor d/2 \rfloor \quad \triangleright$  Number assigned 10 at the top
15:  $q_L \leftarrow \lceil \frac{v_L}{2} \rceil, q_R \leftarrow \lfloor \frac{v_R}{2} \rfloor$ 
16:  $r_D \leftarrow 0, r_L \leftarrow 0, r_R \leftarrow 0$ 
17: Create length- $\ell_g$  arrays  $C_{\text{cur}}, T_{\text{top}}$ , and  $T_{\text{bot}}$ 
18: for  $i \leftarrow 0 \dots \ell_g - 1$  do
19:    $x \leftarrow T[2i], y \leftarrow T[2i + 1]$ 
20:    $M \leftarrow \text{TAGPAIRMASK}(x, y)$ 
21:    $\text{is}_D \leftarrow M[5] \wedge \llbracket r_D < q_D \rrbracket$ 
22:    $\text{is}_L \leftarrow (M[1] \vee M[3] \vee M[8]) \wedge \llbracket r_L < q_L \rrbracket$ 
23:    $\text{is}_R \leftarrow (M[2] \vee M[3] \vee M[6]) \wedge \llbracket r_R < q_R \rrbracket$ 
24:    $\text{fixed}_L \leftarrow M[4] \vee M[6] \vee M[9]$ 
25:    $\text{fixed}_R \leftarrow M[7] \vee M[8] \vee M[9]$ 
26:    $t_L \leftarrow \text{fixed}_L \vee \text{is}_D \vee \text{is}_L$ 
27:    $t_R \leftarrow \text{fixed}_R \vee (M[5] \wedge \neg \text{is}_D) \vee \text{is}_R$ 
28:    $b_L \leftarrow x[0] + y[0] - t_L$ 
29:    $b_R \leftarrow x[1] + y[1] - t_R$ 
30:    $t_{\text{top}} \leftarrow (t_L, t_R)$ 
31:    $t_{\text{bot}} \leftarrow (b_L, b_R)$ 
32:    $C_{\text{cur}}[i] \leftarrow \text{OFORKCONTROL}(x, y, t_{\text{top}}, t_{\text{bot}})$ 
33:    $T_{\text{top}}[i] \leftarrow t_{\text{top}}$ 
34:    $T_{\text{bot}}[i] \leftarrow t_{\text{bot}}$ 
35:    $r_D \leftarrow r_D + M[5]$ 
36:    $r_L \leftarrow r_L + (M[1] \vee M[3] \vee M[8])$ 
37:    $r_R \leftarrow r_R + (M[2] \vee M[3] \vee M[6])$ 
38: end for
39: return  $(C_{\text{cur}}, T_{\text{top}}, T_{\text{bot}})$ 
```
